Optimization in Deep Learning

Concepts and Hands-On Exercises

O. Olawale Awe, PhD
AIMS Cameroon Deep Learning Course 2025

December 5, 2025

Revision: Automatic Feature Extraction in Deep Learning

Deep networks uses all input features, but also:

- Learn **new internal features** in hidden layers.
- Each hidden neuron computes:

$$h_j = \sigma(w_j^\top x + b_j)$$

- These h_j are **learned combinations** of inputs:
  - Non-linear transformations
  - Interactions between variables
  - Higher-level patterns in data

Pipeline: Raw data $\rightarrow$ Layer 1 (features) $\rightarrow$ Layer 2 (richer features) $\rightarrow$ Output

Example: Images (Why Depth Helps)

Classical ML on images:

- You must hand-craft features:
  - Edges, corners, textures, HOG, SIFT, etc.

Deep Learning (e.g. CNNs):

- Layer 1 learns: edges and simple patterns.
- Layer 2 learns: corners, shapes, textures.
- Deeper layers learn: object parts and whole objects.

This hierarchy is learned automatically from data.

Key Question in Deep Learning

Question 1: "Does deep learning not still use all input features like machine learning?"

- **Yes**, it uses all input features.
- **But in addition**, it:
  - Builds new features layer by layer.
  - Learns non-linear combinations without manual design.

Summary:

- Classical ML: you design features, model is shallow.
- Deep Learning: model learns features, model is deep.

Visual Comparison (Conceptual)

Classical ML	**Deep Learning**
Raw data	Raw data
↓	↓
Manual feature engineering	Layer 1: learned features
↓	↓
Shallow model (LR, SVM)	Layer 2: richer features
	↓
	Deep classifier (MLP, CNN, etc.)

Automatic feature extraction = letting the network learn these intermediate representations.

Hidden Neurons Create New Features (Numeric Example)

Consider two raw input features:

$$x_1 = 2, \qquad x_2 = -1$$

A hidden neuron computes:

$$h = \sigma(w_1 x_1 + w_2 x_2 + b)$$

Suppose:

$$w_1 = 0.5, \quad w_2 = -1.2, \quad b = 0.3$$

Compute:

$$z = 0.5(2) + (-1.2)(-1) + 0.3 = 1.0 + 1.2 + 0.3 = 2.5$$

Activation (ReLU):

$$h = \max(0,\, 2.5) = 2.5$$

This hidden neuron has created a new feature:

$$h = 2.5$$

It represents a learned transformation of (x_1, x_2) that did not exist before.

Interpretation of the New Feature

The hidden neuron learned:

$$h = 0.5x_1 - 1.2x_2 + 0.3$$

Meaning:

- It is combining the raw inputs in a non-linear way.
- The model discovered this transformation automatically.
- This feature may represent:
  - a pattern,
  - an interaction,
  - a boundary,
  - or a direction in feature space that matters for prediction.
- Classical ML cannot create this without human feature engineering.

Deep learning = automated creation of new useful features.

Question 2: What is an Epoch? (Quick Explanation)

- An **epoch** is one full pass of the training dataset through the model.
- One epoch consists of many **batches**, each producing one gradient update.
- Multiple epochs help the model gradually reduce error and learn patterns.

Example: Dataset size = 1,000 samples, Batch size = 100

$$\text{Iterations per epoch} = 1000/100 = 10.$$

Epoch is a major hyperparameter in deep learning. Hyperparameters determine how the model learns; parameters determine what the model learns.

Parameters vs Hyperparameters

Parameters (learned during training):

- Weights of neural network layers
- Bias terms
- Embedding vectors
- Convolutional filters

$$\theta = \{W_1, b_1, W_2, b_2, \dots\}$$

They are updated by:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

Hyperparameters (set by the user, not learned):

- Learning rate
- Batch size
- Number of epochs
- Number of layers, neurons
- Dropout rate
- Optimizer type (Adam, SGD)

Question 3: What is Optimization in Deep Learning?

Optimization is the process of automatically adjusting neural network parameters to minimize error using gradient-based algorithms. **It is the process of teaching the network how to learn.**

- Training a neural network means **minimizing the loss function**.
- We update weights using gradient-based algorithms.
- The optimizer determines:
  - How fast we learn,
  - Whether we converge,
  - Whether we escape local minima,
  - Training stability.

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial L}{\partial w}$$

Why Do We Need Optimizers?

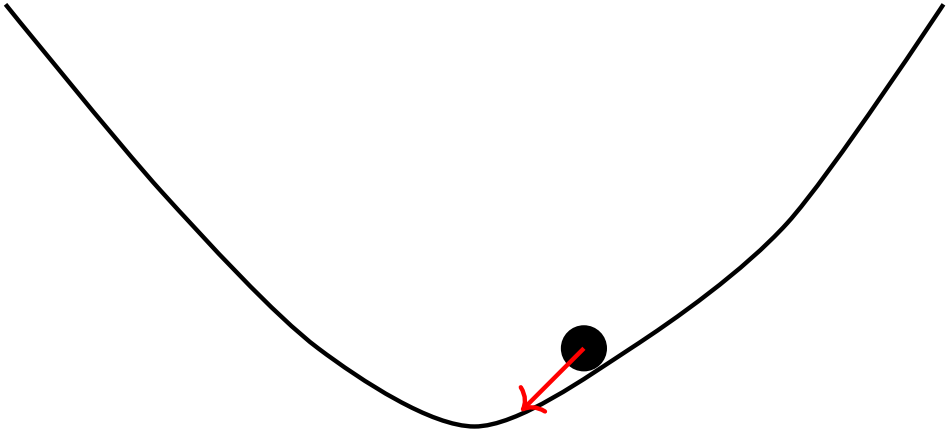
Optimizers determine:

- How weights are updated
- How quickly the model learns
- Whether training converges or diverges
- How to handle noisy or unstable gradients

General update rule:

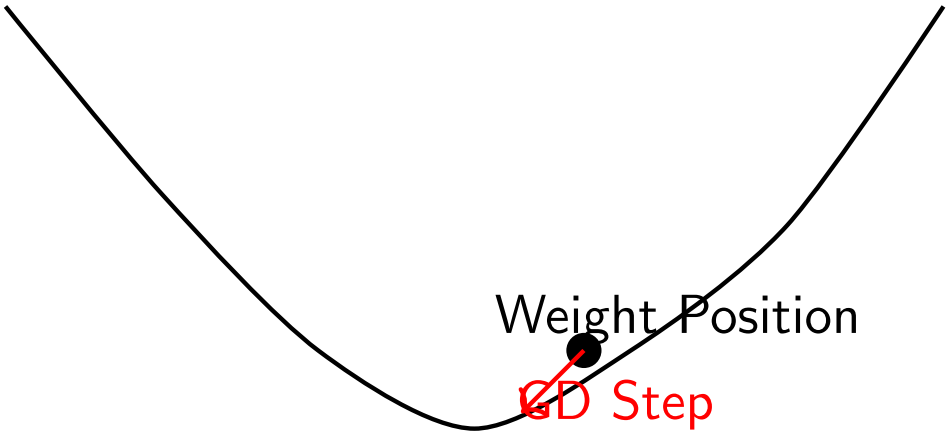
$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

Analogy: Climbing Down a Mountain



- **Optimizer** = walking technique
- **Learning rate** = length of each step

Gradient Descent Intuition (Diagram)



Gradient points uphill $\rightarrow$ we move downhill.

Learning Rate (α)

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$$

- **Too small:** slow learning, stuck in plateaus.
- **Too large:** oscillation, divergence.
- Good training requires careful tuning of α.

Learning rate is the single most important hyperparameter in Optimization

Gradient Descent Variants

Batch Gradient Descent

- Uses all data at once.
- Stable but slow.

Stochastic Gradient Descent (SGD)

- Uses one sample at a time.
- Fast but noisy.

Mini-Batch Gradient Descent

- Uses small batches (e.g., 32 or 64).
- Most common method in deep learning.

Stochastic Gradient Descent (SGD)

Update rule:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

Pros:

- Simple
- Efficient for large datasets

Limitations:

- Can oscillate
- Slow convergence in ravines

SGD with Momentum

Momentum introduces a **velocity term** that accumulates past gradients.

$$v_t = \beta v_{t-1} + (1 - \beta)\nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

Interpretation:

- v_t: exponentially smoothed gradient history
- β: momentum coefficient (0.0–0.999)
- η: learning rate
- θ: model parameters

Effect:

- Speeds up learning in useful directions
- Reduces oscillation in steep ravines
- Helps escape shallow minima and saddle points

Intuition: Why Momentum Works

Scenario: Optimization in a ravine

- One direction has steep curvature (zig-zag risk)
- One direction is shallow (slow progression)
- Plain SGD oscillates heavily

Momentum effect:

- Averages the gradient direction over time
- Reduces oscillations in steep directions
- Accelerates motion along the shallow direction

Analogy: Rolling a ball downhill, once velocity builds, it keeps going unless slopes change drastically.

SGD with Momentum: Pros and Cons

Advantages

- Faster convergence than vanilla SGD
- Reduces oscillations in narrow valleys
- Helps escape saddle points and local minima
- Handles noisy gradients better

Disadvantages

- Needs tuning of an extra hyperparameter (β)
- May overshoot minima when momentum is too strong
- Can become unstable with large learning rates
- Does not adapt step size based on geometry (unlike Adam)

Momentum vs Plain SGD

Property	**SGD**	**SGD + Momentum**
Gradient smoothing	No	Yes
Convergence speed	Slow	Faster
Oscillations in ravines	High	Low
Escape saddle points	Weak	Strong
Extra hyperparameters	None	β

How to Choose Hyperparameters

Typical default settings:

- $\beta = 0.9$ (commonly used)
- η: slightly higher than for plain SGD

Practical guidelines:

- If gradients oscillate $\rightarrow$ decrease η or increase β
- If convergence is slow $\rightarrow$ increase η
- If training diverges $\rightarrow$ reduce both η and β

Tip: Momentum is particularly effective on deep networks and curved loss surfaces.

Why Use Momentum?

- Uses historical gradients to create a velocity
- Smooths noisy updates and stabilizes training
- Accelerates convergence, especially on long, flat valleys
- Reduces zig-zag motion in steep directions
- A simple but powerful improvement over vanilla SGD

RMSProp

RMSProp adapts the learning rate **independently per parameter** by keeping an exponentially decaying average of squared gradients.

$$r_t = \rho r_{t-1} + (1-\rho)\, g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t + \epsilon}}\, g_t$$

Key Concepts:

- g_t: gradient at time t
- r_t: running average of squared gradients
- ρ: decay factor (0.9–0.99 typically)
- η: learning rate
- ϵ: small constant to avoid division by zero

Why RMSProp Works

Effect of the Denominator:

$$\frac{\eta}{\sqrt{r_t + \epsilon}}$$

- If gradients are large $\Rightarrow r_t$ increases, learning rate decreases.
- If gradients are small $\Rightarrow r_t$ decreases, learning rate increases.
- Each parameter gets its own adaptive step size.

Benefits:

- Handles non-stationary objectives
- Prevents exploding updates
- Works extremely well for RNNs and sequence models

Understanding the Parameters

Decay Rate ρ:

- Typical default: $\rho = 0.9$
- Controls how much past gradients influence the running average
- Higher ρ = smoother average, slower adaptation

Learning Rate η:

- Often chosen slightly smaller than for Adam
- RMSProp adapts LR but still depends on a good base value

Stability Term ϵ:

- Prevents division by zero
- Improves numerical stability

RMSProp Intuition

Imagine a ravine-shaped loss surface:

- One direction has large gradients
- Another direction has small gradients

RMSProp Action:

- Scales down learning rate in steep directions
- Scales up learning rate in flat directions
- Reduces oscillations and speeds convergence

Analogy: Walking downhill with adaptive steps, the steeper the slope, the smaller your step automatically becomes.

Pros and Cons of RMSProp

Advantages

- Adapts learning rate per parameter
- Handles noisy or non-stationary gradients
- Very effective for RNNs and deep sequence models
- Avoids the diminishing learning rate problem of AdaGrad

Disadvantages

- Extra hyperparameters (ρ, ϵ) must be tuned
- Still sensitive to the base learning rate η
- Does not include momentum by default (Adam does)

RMSProp vs Other Optimizers

Property	**SGD**	**AdaGrad**	**RMSProp**
Adaptive LR	No	Yes (shrinks too fast)	Yes (stable)
Handles non-stationary gradients	Weak	Moderate	Strong
Uses momentum	No	No	Optional
Best use case	Convex problems	Sparse features	RNNs, deep nets

When to Use RMSProp

RMSProp is especially suitable when:

- Training Recurrent Neural Networks (LSTMs, GRUs)
- Gradients vary over time (non-stationary)
- Loss landscape contains steep and flat regions
- You want AdaGrad-like adaptivity without shrinking LR too quickly

Why Choose RMSProp?

- Maintains a running average of squared gradients
- Produces adaptive learning rates per parameter
- Prevents learning rate collapse (unlike AdaGrad)
- Excels on RNNs and dynamically changing tasks
- A cornerstone of modern optimization (basis for Adam)

Adam Optimizer

Adam = Momentum + RMSProp

Adam maintains:

- a running average of gradients (first moment)
- a running average of squared gradients (second moment)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias-corrected estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Parameter update:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Understanding the Parameters

Hyperparameters:

- β_1 (default 0.9): controls smoothing of gradients (momentum)
- β_2 (default 0.999): controls smoothing of squared gradients (RMSProp)
- η (default 0.001): step size / learning rate
- ϵ (10^{-8}): avoids division by zero

Interpretation:

- m_t: direction of motion (like velocity)
- v_t: adaptive scaling (controls step size)
- Adam uses **both** momentum and per-parameter adaptivity

Why Adam Works

Adam combines the strengths of:

- **Momentum:** smoothing of gradients $\rightarrow$ stable direction of descent
- **RMSProp:** adaptive learning rate per parameter $\rightarrow$ handles curvature

Benefits:

- Works very well with noisy gradients
- Fast convergence in practice
- Less need for manual learning rate tuning
- Performs well across many neural network architectures

Reason it's default: Adam is robust, efficient, and requires minimal hyperparameter adjustment.

Bias Correction: Why It Matters

At early steps, m_t and v_t are biased toward zero.
Adam fixes this with:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Effect of bias correction:

- Prevents overly small updates at the start
- Stabilizes early learning
- Ensures consistent scaling of gradients

Pros and Cons of Adam

Advantages

- Fast, stable convergence
- Handles sparse and noisy gradients well
- Excellent for deep networks (CNNs, RNNs, Transformers)
- Minimal tuning required
- Adaptive per-parameter scaling

Disadvantages

- Sometimes converges to slightly worse minima than SGD
- Can overfit if learning rate is too high
- More computation than vanilla SGD
- Not always ideal for large-scale convex optimization

Adam vs Other Optimizers

Property	**SGD**	**Momentum**	**RMSProp**	**Adam**
Adaptive LR	No	No	Yes	Yes
Momentum term	No	Yes	No	Yes
Bias correction	No	No	No	Yes
Good for RNNs	Weak	Moderate	Strong	Very Strong
Default for DL	No	No	No	Yes
Hyperparameters	η	η, β	η, ρ	η, β_1, β_2

When to Use Adam

Adam is especially useful for:

- Deep Neural Networks (CNNs, RNNs, Transformers)
- Problems with noisy gradients (e.g., mini-batch training)
- Sparse features (NLP, text embeddings)
- Training very deep or complex models
- Non-convex optimization landscapes

General rule: Use Adam by default unless you have reason to prefer SGD.

Summary: Why Adam is the Default Choice

- Combines momentum and adaptive learning rates
- Fast, reliable convergence on most deep learning tasks
- Works well out of the box with default parameters
- Handles noisy and sparse gradients extremely well
- Stable training process even in deep architectures

Bottom Line: Adam = Practical, Efficient, Robust.

Loss Landscapes and Convergence

Deep networks optimize highly non-convex functions.
Challenges:

- Local minima
- Saddle points
- Vanishing or exploding gradients

Optimizers like Momentum and Adam help navigate complex landscapes.

AdaGrad Optimizer (Adaptive Gradient Algorithm)

Idea: AdaGrad adapts the learning rate for each parameter based on the *history of squared gradients*. Frequently updated parameters receive smaller learning rates, while rare features get larger ones.

Update Mathematics

Let g_t be the gradient at step t for parameter θ.

$$G_t = G_{t-1} + g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t$$

Where:

- η = initial learning rate,
- G_t = accumulated squared gradients,
- ϵ = numerical stability term.

AdaGrad Intuition

Key Idea

- Parameters with large gradients $\Rightarrow$ smaller learning rate.
- Parameters with rare/small gradients $\Rightarrow$ larger learning rate.
- Works well for sparse data (e.g., NLP, text).

Limitation

G_t grows without bound $\Rightarrow$ learning rates shrink to nearly zero.
This causes very **slow convergence** or early stopping of learning.

AdaGrad vs RMSProp vs Adam

AdaGrad	Accumulates all past squared gradients forever. Learning rate becomes extremely small over time.
RMSProp	Uses exponential moving average of squared gradients; prevents the learning rate from shrinking to zero.
Adam	RMSProp + Momentum. Adaptive learning rate + velocity term. Most widely used optimizer today.

Key idea: AdaGrad introduced adaptive learning rates, but RMSProp and Adam fixed AdaGrad's shrinking-learning-rate problem.

Summary Comparison of Major Optimization Concepts

Optimizer	Key Feature	Best For
SGD	Simple, baseline	Small models
Momentum	Smooth, fast	Deep networks
RMSProp	Adaptive rates	RNNs
Adam	Fast + adaptive	Most tasks

Find out and learn about other methods from the references below.

Exercise 1: Manual SGD Step

Given:

$$f(w) = (w-4)^2$$

$$f'(w) = 2(w-4)$$

Start: $w_0 = 0$, LR $= 0.1$

Compute:

$$w_1 = 0 - 0.1(-8) = 0.8$$

$$w_2 = 0.8 - 0.1(-6.4) = 1.44$$

$$w_3 = 1.44 - 0.1(-5.12) = 1.952$$

$$\boxed{w \to 4}$$

Exercise 2: Momentum by Hand

$$v_t = \beta v_{t-1} + g_t \quad \beta = 0.9$$

Gradients:

$$g_1 = 5,\ g_2 = 3,\ g_3 = 1$$

$$v_1 = 5, \quad v_2 = 7.5, \quad v_3 = 7.75$$

Momentum accelerates updates.

Exercise 3: RMSProp

$$r_t = 0.9 r_{t-1} + 0.1 g_t^2$$

Given:

$$r_0 = 0, \;\; g_1 = 4, \;\; g_2 = 2$$

$$r_1 = 1.6, \qquad r_2 = 1.84$$

Exercise 4: Adam Calculation

$$m_t = 0.9m_{t-1} + 0.1g_t \quad v_t = 0.99v_{t-1} + 0.01g_t^2$$

$$g_1 = 5$$

$$m_1 = 0.5, \quad v_1 = 0.25$$

Exercise 5: Step Sizes Across Optimizers

Suppose we optimize a single parameter θ.

- Current gradient: $g = 2.0$
- Learning rate: $\eta = 0.1$
- Initial states: $v_0 = 0,\ r_0 = 0,\ m_0 = 0,\ \theta_0$ arbitrary

Tasks:

1. Compute the update for:
   - SGD: $\theta_1 = \theta_0 - \eta g$
   - Momentum ($\beta = 0.9$): $v_1 = 0.9v_0 + 0.1g$
   - RMSProp ($\rho = 0.9$): $r_1 = 0.9r_0 + 0.1g^2$
   - Adam ($\beta_1 = 0.9,\ \beta_2 = 0.999$): compute m_1, v_1 and bias-corrected $\hat{m}_1, \hat{v}_1$.
2. Compare the effective step sizes of each optimizer.

Exercise 6: Oscillation vs Stability in a Steep Valley

We are in a steep valley where gradients flip sign:

$$g_1 = 5, \quad g_2 = -5$$

Hyperparameters:

- Learning rate: $\eta = 0.1$
- Momentum: $\beta = 0.9$
- RMSProp: $\rho = 0.9$

Tasks:

1. Compute two SGD updates and show the oscillation in θ.
2. Compute two Momentum updates (with $v_0 = 0$) and observe how oscillation is reduced.
3. Compute two RMSProp updates (with $r_0 = 0$) and observe how the step size adapts.
4. Discuss which method is most stable in this steep valley.

Exercise 7: Noisy Gradients (RMSProp & Adam)

Consider a noisy gradient sequence:

$$g_1 = 1.0,\ g_2 = 0.2,\ g_3 = 3.0,\ g_4 = 0.1,\ g_5 = 2.0$$

Hyperparameters:

- RMSProp: $\rho = 0.9$
- Adam: $\beta_1 = 0.9,\ \beta_2 = 0.999$

Tasks:

1. Compute r_t for RMSProp for $t = 1, \ldots, 5$.
2. Identify steps where RMSProp shrinks the effective learning rate.
3. Compute v_t (second moment) for Adam for $t = 1, \ldots, 5$.
4. Identify at which steps Adam strongly down-weights the update.
5. Discuss which optimizer behaves more stably under noisy gradients.

Exercise 8: Momentum Building Over Time

Assume a constant gradient:

$$g_t = 1.0 \quad \text{for all } t.$$

Hyperparameters:

- Momentum coefficient: $\beta = 0.9$
- Learning rate: $\eta = 0.1$
- Initial velocity: $v_0 = 0$

Tasks:

1. Compute v_1, v_2, v_3, v_4 using

$$v_t = \beta v_{t-1} + (1 - \beta) g_t.$$

2. Compute the corresponding updates $\Delta\theta_t = \eta v_t$ for $t = 1, \ldots, 4$.
3. Compare the step sizes over iterations and explain how momentum "accelerates" learning even with constant gradients.

Exercise 9: RMSProp vs Adam on Sparse Gradients

A parameter has mostly zero gradients with an occasional spike:

$$g_1 = 0,\ g_2 = 0,\ g_3 = 5,\ g_4 = 0,\ g_5 = 0$$

Use standard hyperparameters:

- RMSProp: $\rho = 0.9$
- Adam: $\beta_1 = 0.9,\ \beta_2 = 0.999$
- Assume initial states $r_0 = 0,\ m_0 = 0,\ v_0 = 0$.

Tasks:

1. Compute r_t for RMSProp for $t = 1, \ldots, 5$ and describe its behavior near the spike at $t = 3$.
2. Compute v_t (second moment) for Adam for $t = 1, \ldots, 5$.
3. Compare how RMSProp and Adam adapt after the spike at $t = 3$.
4. Discuss why Adam is often preferred for sparse gradients (e.g. in NLP).

Exercise 10: Which Optimizer Reaches the Minimum Faster?

We minimize the 1D function:

$$f(\theta) = (\theta - 3)^2, \quad \nabla f(\theta) = 2(\theta - 3).$$

Start from:

$$\theta_0 = 10.$$

Consider:

- SGD with $\eta = 0.1$
- Momentum with $\eta = 0.1,\ \beta = 0.9,\ v_0 = 0$
- RMSProp with $\eta = 0.1,\ \rho = 0.9,\ r_0 = 0$
- Adam with $\eta = 0.1,\ \beta_1 = 0.9,\ \beta_2 = 0.999,\ m_0 = v_0 = 0$

Tasks:

1. Perform 3 iterations for each optimizer (by hand or in a small table).
2. Track the sequence $\theta_0, \theta_1, \theta_2, \theta_3$.
3. Compare how quickly each optimizer moves θ towards the minimizer $\theta^* = 3$.

Exercise 11: Large vs Small Gradients

Consider two gradient magnitudes at different steps:

$$g_1 = 0.1, \quad g_2 = 5.0.$$

Use the same base learning rate η for all optimizers.
Tasks:

1. For RMSProp, compute r_1 and r_2, and the corresponding effective learning rates $\eta/\sqrt{r_1}$ and $\eta/\sqrt{r_2}$.
2. For Adam, compute v_1 and v_2 (second moments) and discuss how they affect the denominator in the update.
3. Compare how RMSProp and Adam penalize large gradients relative to small gradients.
4. Explain why this penalty on large gradients is helpful in deep learning training.

Exercise 12: Fill-in-the-Table (All Optimizers Together)

Fill in the missing entries for each optimizer. Assume:

- Learning rate: $\eta = 0.1$
- Momentum: $\beta = 0.9$
- RMSProp: $\rho = 0.9$
- Adam: $\beta_1 = 0.9,\ \beta_2 = 0.999$
- Initial states: $v_0 = 0,\ r_0 = 0,\ m_0 = 0,\ v_0^{(Adam)} = 0$

Step	g_t	SGD update	Momentum v_t	RMSProp r_t	Adam m_t	Adam v_t
1	1.5	?	?	?	?	?
2	0.5	?	?	?	?	?
3	-1.0	?	?	?	?	?

Task 12: Complete the table and compare the behavior of each optimizer over the 3 steps.

More Exercises

Task 13 : Explore Epochs

- Run the PyTorch/TensorFlow code.
- Try various epochs: **1, 5, 20, etc**.
- Observe training loss and signs of overfitting.

Task 14 : Explore Hyperparameters

- Change batch size: **8, 32, 128**.
- Change learning rate: **0.1, 0.01, 0.001**.
- Compare learning curves for each combination.

Task 15 : Write a Short Report

- In your experiment, summarize how changing epochs, batch size, optimization method or learning rate affected:
  - convergence speed
  - stability of loss
  - model performance

References

Fleuret, F. (2024).
The Little Book of Deep Learning.
University of Geneva.

Goodfellow, I., Bengio, Y., Courville, A. (2016).
Deep Learning.
MIT Press, Cambridge.

Bishop, C. M., & Bishop, H. (2023).
Deep Learning: Foundations and Concepts.
Springer Nature.

Prince, S. J. (2023).
Understanding Deep Learning.
MIT Press.